

2026 年 5 月 8 日 Version 1.0

ハッカソン Processing 音源サブ システム 基本設計・詳細設計

チーム 23
25G1021 : 梅澤 颯太

目次

第 1 章	システムの基本設計	1
1.1	機能一覧	1
1.2	システム構成図	2
1.2.1	パート対応	2
1.3	操作インターフェース設計	3
1.4	状態遷移図	4
第 2 章	システムの詳細設計	7
2.1	部品一覧	7
2.2	ハードウェア設計	7
2.3	ソフトウェア設計	8
2.3.1	ファイル構成	9
2.3.2	NOTE パケット設計	9
2.3.3	オブジェクト設計	10
2.3.4	音色設計	12
2.4	処理フローの設計	13
2.4.1	起動フロー	13
2.4.2	NOTE 受信フロー	14
2.4.3	ボイス管理フロー	14
2.4.4	例外処理	15
2.5	テストの設計	15
2.6	ループリック対応	16
第 3 章	生成 AI の利用について	18

第 1 章 システムの基本設計

1.1 機能一覧

Processing 音源サブシステムの機能一覧を表 1.1 に示す。本書では Processing 側の機能を F6 として定義し、塩澤設計の F4「発音情報配信」の受け口以降を担当範囲とする。

表 1.1: Processing 音源サブシステムの機能一覧

機能	役割
Serial 接続管理	Processing 起動時に USB Serial ポートを開く。切断時は音を止め、ユーザ操作または自動再試行で再接続できる状態へ戻す。
NOTE フレーム同期・復号	Serial から届くバイト列をバッファへ蓄積し、magic=0x4F52 を探して 20 バイト単位に切り出す。version/type/seq/partId を検査する。
パート設定管理	partId=0x02--0x05 と音色・発音枠を対応付ける。本番は各 Mac が 1 パートを担当し、検証時は 1 台で 4 パートを受けられる統合モードも持つ。
ボイス管理	NoteOn, NoteOff, durationMs 経過による自動停止、同時発音数上限、古いボイスの解放を管理する。
音色生成	金管パートは倍音加算合成と ADSR で音色を作る。リズム伴奏パートは短い減衰を持つ打音系として扱い、塩澤・齋藤の node_05 仕様と矛盾しないようにする。

機能	役割
画面表示・操作 IF	接続状態, 期待 partId, 受信 seq, 発音中ボイス数, 警告, ミュート, テスト発音を表示・操作する.
ログ・検証支援	受信時刻, 復号結果, 破棄理由, 発音開始時刻を CSV に残し, 結合試験で遅延と欠落を評価できるようにする.

1.2 システム構成図

図 1.1 に, Processing 音源サブシステムを含む全体構成を示す. 塩澤設計との整合を優先し, 各楽器ノードは専用 Mac と USB Serial で 1 対 1 接続する. ただし, Processing の内部設計は 4 パートを扱える形にしており, 結合前の検証では 1 台の Mac で全パートを再生できるようにする.

Listing 1.1 Processing 音源サブシステムを含む全体構成

```
User conducts
  |
  v
node_01: conductor, CTRL/BEAT sender
  |
  | UDP multicast CTRL/BEAT
  v
node_02 -- USB Serial NOTE --> Mac 1 Processing --> Brass 1
node_03 -- USB Serial NOTE --> Mac 2 Processing --> Brass 2
node_04 -- USB Serial NOTE --> Mac 3 Processing --> Brass 3
node_05 -- USB Serial NOTE --> Mac 4 Processing --> rhythm/drum
```

1.2.1 パート対応

Processing は表 1.2 の対応で partId を解釈する. 齋藤設計の楽譜上の役割と, 塩澤設計のノード ID をつなぐ境界であるため, 本表は実装時にも設定値として残す.

表 1.2: partId と音色・役割の対応

ノード	partId	楽譜上の役割	Processing 側音色
node_02	0x02	主旋律	金管 1. 基準となる BrassVoice.
node_03	0x03	輪唱パート 1	金管 2. BrassVoice の倍音比率は同じで、音量と定位を変えられる設計にする.
node_04	0x04	輪唱パート 2	金管 3. BrassVoice の設定を共有し、必要なら少し明るい音色に調整する.
node_05	0x05	ベース・リズム伴奏	短い減衰を持つ Drum/RhythmVoice. 金管音色で無理に鳴らさない.

1.3 操作インターフェース設計

Processing のユーザは演奏者ではなく、デモ準備者または結合試験担当者である。観客や指揮者は Processing を直接操作しない。そのため、画面とキー操作は演奏中の障害発見と安全停止に絞る。

表 1.3: Processing 側操作インターフェース

操作・表示	対象	システム挙動
起動	Mac 担当者	Serial ポート一覧を取得し、設定済みポートがあれば接続を試みる.

操作・表示	対象	システム挙動
1-4 キー	Mac 担当者	期待 partId を 0x02-0x05 に切り替える. 本番では Mac ごとに固定する.
a キー	検証担当者	統合モードに入り, 0x02-0x05 の全パートを受け付ける.
r キー	Mac 担当者	Serial を閉じ, ポート選択待ちへ戻る. USB 抜けや誤接続の復旧に使う.
m キー	Mac 担当者	ミュートを切り替える. ミュート中も NOTE 復号とログ記録は続ける.
t キー	Mac 担当者	選択中 partId のテスト音を鳴らし, 音量とスピーカ接続を確認する.
画面上部	全員	接続状態, ポート名, 期待 partId, 直近 seq, 欠落数, 破棄数を表示する.
画面下部	全員	発音中ボイス数, ミュート状態, 波形モニタ, 直近エラーを表示する.

1.4 状態遷移図

Processing 側の状態遷移を図 1.2 に示す. 状態は UI 表示, Serial 接続, 発音管理の 3 つを一貫して制御するために使う. Serial 例外時に発音が残り続けないことを重視する.

Listing 1.2 Processing 音源サブシステムの状態遷移

Boot

```

-> PortSelect : setup completed
PortSelect
-> Ready      : serial port opened
-> Error      : port open failed
Ready
-> Playing    : first valid NOTE received
-> PortSelect : user presses r
Playing
-> Muted      : user presses m
-> Ready      : no NOTE for 1000 ms, serial still alive
-> Error      : serial exception or timeout > 3000 ms
Muted
-> Playing    : user presses m
-> Error      : serial exception
Error
-> PortSelect : user presses r or retry interval elapsed

```

表 1.4: 状態定義

状態	意味	主な処理
Boot	起動直後	画面, 音声出力, 設定値, ログ, 内部配列を初期化する.
PortSelect	Serial ポート選択待ち	利用可能ポートを表示し, ユーザ選択または保存設定で接続する.
Ready	接続済み・NOTE 待ち	Serial バッファを空にし, フレーム同期を開始する.
Playing	正常受信・発音中	NOTE 復号, partId 検査, Voice 生成, 画面更新, ログ出力を行う.
Muted	受信継続・発音停止	NOTE は処理してログに残すが, 新規 Voice を作らない. 既存 Voice は Release へ移す.

状態	意味	主な処理
Error	接続異常・復号不能	Serial を閉じ，全 Voice を停止し，エラー理由を表示する.

第 2 章 システムの詳細設計

2.1 部品一覧

Processing 側に関係する構成要素を表 2.1 に示す。本書の主対象はソフトウェアであるが、ループリック上のハードウェア設計項目に対応するため、接続関係と動作も明確にする。

表 2.1: Processing 側関連部品

ID	要素	数量	役割
P1	Arduino UNO R4 WiFi	4	node_02-05. 楽譜進行後, NOTE パケットを USB Serial で送信する.
P2	Mac または PC	4	Processing 実行機. 本番は 1 台につき 1 つの Serial ポートを開く.
P3	USB Type-C ケーブル	4	Arduino と Mac の給電および Serial 通信を兼ねる.
P4	スピーカまたは内蔵スピーカ	4	Processing の音声出力を再生する. 音量は結合時に手動調整する.
P5	Processing Sound/Minim ライブラリ	4 + 4 環境系	NOTE を受けて音を合成する実行環境. 授業演習で扱った Processing 音生成を発展させる.

2.2 ハードウェア設計

接続は表 2.2 の通りとする。Processing 側では回路を追加しないが、Serial 接続と音声出力の対応を固定し、誤接続時に発音しない設計にする。

表 2.2: Processing 側ハードウェア接続設計

項目	設計
接続方式	node_02-05 と Mac 1-4 を USB Type-C で 1 対 1 接続する。
通信方式	USB CDC Serial, Arduino 側は <code>Serial.write()</code> で NOTE を送る。
ボーレート	115200 bps. USB CDC では実効速度に余裕があるが、Arduino 側設定と統一する。
フレーミング	全 NOTE は先頭 2 バイトの <code>magic=0x4F52</code> を同期マークとする。Serial 上ではリトルエンディアンで <code>0x52 0x4F</code> の順に現れる。
音声出力	各 Mac の既定オーディオ出力へ送る。本番は OS 音量をそろえ、クリップがない範囲で調整する。
誤接続対策	Mac ごとに期待 <code>partId</code> を設定し、異なる <code>partId</code> を受信した場合は発音せず警告とログを残す。
切断時動作	USB 切断または Serial 例外を検出したら Error へ遷移し、全 Voice を停止する。

2.3 ソフトウェア設計

2.3.1 ファイル構成

実装時の配置案を図 2.1 に示す。本書は設計書でありファイルを実装しないが、クラス責務を明確にするため、Processing スケッチの分割単位を定義する。

Listing 2.1 Processing 実装予定ファイル構成

```
pc_app/
`-- orchestra_processing/
    |-- orchestra_processing.pde    // setup(), draw(), keyPressed()
    |-- Config.pde                  // expected partId, serial, audio
        params
    |-- OrcProtocol.pde              // NotePacket and little-endian
        decoder
    |-- SerialFrameReader.pde        // byte buffer and magic sync
    |-- PartManager.pde              // part filter and voice allocation
    |-- Voice.pde                    // BrassVoice, RhythmVoice, ADSR
    |-- ToneFactory.pde              // harmonic tables and waveform
        creation
    |-- UIView.pde                  // status screen and waveform monitor
    `-- Logger.pde                  // CSV logging
```

2.3.2 NOTE パケット設計

Processing は表 2.3 の 20 バイトを固定長 NOTE として読む。この表は塩澤設計の受信側解釈であり、Processing 側で独自に別形式を定義しない。

表 2.3: NOTE パケットの受信側解釈

offset	field	type	Processing 側の扱い
0	magic	uint16	0x4F52 以外なら同期探索を続ける。
2	version	uint8	0x01 以外なら破棄し、version error としてログする。

offset	field	type	Processing 側の扱い
3	type	uint8	3 のときだけ NOTE として処理する. CTRL/BEAT は Processing で発音しない.
4	seq	uint32	前回値との差から重複・欠落を検出する.
8	timestampMs	uint32	Arduino 側の送信時刻. 遅延評価用にログする.
12	partId	uint8	0x02-0x05. 期待 partId または ALL と一致する場合だけ発音する.
13	noteNumber	uint8	MIDI ノート番号として周波数へ変換する. 0 は休符または停止相当として扱う.
14	velocity	uint8	0-127 を振幅へ変換する. 総振幅が過大にならないよう正規化する.
15	gate	uint8	1=NoteOn, 0=NoteOff. その他は破棄する.
16	durationMs	uint16	NoteOn 時の自動停止予定時刻を決める. 0 の場合は最小長を使う.
18	reserved	uint8[2]	0 以外でも発音は継続するが警告としてログする.

2.3.3 オブジェクト設計

表 2.4 に主要クラスを示す. クラス名は実装時の案であり, 重要なのは責務分離である. Serial 復号, パート管理, 音色生成, UI を分けることで, 結合試験で問題箇所を切り分けやすくする.

表 2.4: Processing 側オブジェクト設計

クラス・構造体	主なフィールド・メソッド	役割
NotePacket	partId, noteNumber, velocity, gate, durationMs, seq, timestampMs	20 バイトフレームから復号した発音指令を表す.
SerialFrameReader	pushByte(), tryReadPacket(), dropUntilMagic()	Serial 入力を蓄積し, magic 同期と 20 バイト切り出しを行う. 破損時は 1 バイトずつ捨てて再同期する.
PartConfig	expectedPartId, instrumentType, maxVoices	Mac ごとの担当パートと音色を定義する.
PartManager	handleNote(), noteOn(), noteOff(), update()	partId 検査, 同時発音上限, durationMs による自動停止を管理する.
Voice	start(), release(), isFinished()	1 つの発音単位を表す基底クラス. 周波数, 振幅, 開始時刻, 終了予定時刻を持つ.
BrassVoice	harmonics, attackMs, decayMs, sustain, releaseMs	金管 1-3 の音色を生成する. 倍音比率と ADSR を設定値として持つ.
RhythmVoice	noiseAmount, decayMs, pitchDrop	node_05 のベース・リズム伴奏を短い減衰音として生成する.

クラス・構造体	主なフィールド・メソッド	役割
UIView	drawStatus(), drawWaveform(), drawWarning()	接続状態, 受信状況, 発音中 ボイス数, 警告を描画する.
Logger	logPacket(), logError(), flush()	結合試験用 CSV を保存する.

2.3.4 音色設計

2.3.4.1 金管音色

金管音色は、基音に対して複数の倍音を加える加算合成で作る。初期値は表 2.5 とし、授業資料で扱った波形・スペクトル確認や FFT 解析により、実機試験後に調整できるようにする。

表 2.5 金管音色の初期倍音比率

倍音	1	2	3	4	5	6	7
振幅比	1.00	0.70	0.50	0.30	0.20	0.10	0.05

ADSR は表 2.6 の初期値とする。前版のように音符全体を単純な直線減衰にすると、持続音が弱くなり金管らしさが落ちるため、本設計では Attack, Decay, Sustain, Release を分ける。

表 2.6: 金管 ADSR 初期値

項目	初期値	意味	調整方針
Attack	20 ms	息の入りの立ち上がり	遅く感じる場合は 10 ms へ短縮する.
Decay	40 ms	最大音量から持続音量へ 移る時間	音が硬い場合は延長する.

項目	初期値	意味	調整方針
Sustain	最大音量の 0.75 倍	音符中の持続音量	velocity に応じてスケールする.
Release	60 ms	息を止めた後の余韻	音の切れが悪い場合は 40 ms へ短縮する.

2.3.4.2 リズム伴奏音色

node_05 は齋藤資料でベース・リズム伴奏, 塩澤資料でドラムとして扱われる. したがって, Processing 側では金管音色に固定せず, リズムを支える短い減衰音として設計する.

表 2.7: リズム伴奏音色の設計

noteNumber	音色カテゴリ	合成方針
36 付近	Kick/Bass	低いサイン波を短く鳴らし, 100–150 ms 程度で減衰させる.
38 付近	Snare	ノイズ成分と中域の短い共鳴を混ぜ, 90 ms 程度で減衰させる.
42 付近	Hi-hat	高域ノイズを 40 ms 程度で短く減衰させる.
その他	Click	未定義の noteNumber でも無音にならないよう, 短いクリック音で代替する.

2.4 処理フローの設計

2.4.1 起動フロー

1. setup() で画面, 音声出力, 設定値, Logger, PartManager を初期化する.
2. 期待 partId を設定する. 本番では Mac 1–4 が 0x02–0x05 に固定される.

3. `Serial.list()` でポート一覧を取得し、保存済みポートがあれば接続を試みる.
4. 接続成功なら `Ready`, 失敗なら `PortSelect` または `Error` に遷移する.
5. テスト発音でスピーカと音量を確認できるようにする.

2.4.2 NOTE 受信フロー

NOTE 受信から発音までの処理を図 2.2 に示す. ここで重要なのは, Arduino 側が送るバイト列を Processing 側で勝手に文字列化せず, 20 バイト固定長のバイナリとして扱う点である.

Listing 2.2 NOTE 受信から発音までの処理フロー

```
1 serialEvent()  
2     while serial has bytes:  
3         reader.pushByte(serial.read())  
4         while reader.tryReadPacket(packet):  
5             if packet.version != 1:  
6                 log("version error"); continue  
7             if packet.type != 3:  
8                 log("non NOTE packet"); continue  
9             if packet.gate is not 0 and not 1:  
10                 log("gate error"); continue  
11             if expectedPartId != ALL and packet.partId != expectedPartId:  
12                 log("wrong part"); showWarning(); continue  
13             if packet.seq is duplicate:  
14                 log("duplicate seq"); continue  
15             PartManager.handleNote(packet)
```

2.4.3 ボイス管理フロー

1. `gate=1` かつ `noteNumber>0` の場合, `NoteOn` として扱う.
2. `partId` から音色種別を決める. `0x02-0x04` は `BrassVoice`, `0x05` は `RhythmVoice` とする.
3. `velocity` を `0.0-1.0` に正規化し, 合計振幅が `0.8` を超えないようスケールする.
4. 同一 `partId` の発音数が上限を超える場合, 最も古い `Voice` を `Release` へ移す.
5. `durationMs` から自動 `NoteOff` 時刻を計算する. `0` の場合は最小長を使う.
6. `gate=0` または自動 `NoteOff` 時刻到達時に, 同一 `partId` ・ 同一 `noteNumber` の `Voice` を `Release` へ移す.

7. Release が完了した Voice は配列から削除し、音声出力との接続を解除する。

2.4.4 例外処理

表 2.8: Processing 側例外処理

条件	処理	目的
magic 不一致	1 バイトずつ破棄して次の 0x52 0x4F を探す。	バイトずれから復帰 する。
version 不一致	パケット全体を破棄する。	旧仕様・別仕様との混 在を防ぐ。
type 不一致	パケット全体を破棄する。	CTRL/BEAT を誤って 発音しない。
seq 重複	発音せずログのみ残す。	二重発音を防ぐ。
seq 欠落	発音は継続し、欠落数をログ する。	聴感破綻を避けつつ検 証可能にする。
partId 不一致	発音せず画面警告を出す。	Mac と Arduino の接続 ミスを即発見する。
Serial 切断	全 Voice を Release し Error へ遷移する。	無限発音を防ぐ。
音量過大	全 Voice の振幅をスケールダ ウンする。	クリップと耳に痛い音 を防ぐ。

2.5 テストの設計

テストは、Processing 単体、Arduino との結合、音色評価の順に行う。表 2.9 に試験項目を示す。ループリックで要求される「処理フローの条件分岐や例外処理」も確認対象に含める。

表 2.9: Processing 側テスト設計

ID	試験項目	手順	合否基準
PT-1	NOTE 復号単体	正常な 20 バイト配列を与え, NotePacket に復号する.	全フィールドが一致する.
PT-2	フレーム同期復帰	破損バイトを前に混ぜた後, 正常フレームを与える.	次の magic から復帰する.
PT-3	partId 誤接続	Mac 1 設定で partId 0x03 を入力する.	発音せず警告表示する.
PT-4	NoteOn/Off	gate=1 後に gate=0 または durationMs 経過を入力する.	無限発音しない.
PT-5	同時発音上限	金管で上限を超える NoteOn を連続入力する.	古い Voice を Release し, 上限を維持する.
PT-6	リズム伴奏音色	partId 0x05 で複数の noteNumber を入力する.	短いリズム音として鳴る.
PT-7	受信遅延	NOTE 受信時刻から Voice 生成までを 100 回記録する.	最大 5 ms 以下を目標にする.
PT-8	結合演奏	node_02-05 と Mac 4 台を接続し, 4 パートを演奏する.	欠落なく鳴り, 誤接続警告が出ない.

2.6 ルーブリック対応

表 2.10 に, 本書がシステム設計書ルーブリックの評価項目に対応している箇所を示す.

表 2.10: システム設計書ループリックとの対応

評価項目	本書での対応
基本設計: 機能一覧とシステム構成図	第 1 章の機能一覧, 構成図, partId 対応表で主要機能と構成要素の関係を示した.
基本設計: 操作インターフェースと状態遷移	キー操作, 画面表示, Boot から Error までの状態遷移を示した.
詳細設計: ハードウェア設計	Arduino, Mac, USB, スピーカの接続関係と切断時動作を示した.
詳細設計: ソフトウェア設計	ファイル構成, NOTE パケット, クラス設計, 音色パラメータを示した.
詳細設計: 処理フローの設計	起動, 受信, 復号, 発音, 例外処理, テストの流れを示した.

第 3 章 生成 AI の利用について

本設計書の作成にあたり，生成 AI を利用した．利用範囲は，既存資料の読み取り補助，章立ての整理，ループリック項目との対応確認，および文章案の作成である．

ただし，採用する前提条件，NOTE パケット仕様，Mac 4 台構成，partId 対応，齋藤担当範囲との境界，試験項目については，リポジトリ内資料およびチーム内レビューを確認したうえで著者が判断する．生成 AI の出力はそのまま採用せず，塩澤・齋藤の仕様と矛盾しないよう修正した．